ORCA: Real-time Observability and Control for Bulk-Synchronous Parallel Applications

Anonymous Author(s)

ABSTRACT

The *bulk-synchronous parallel* (BSP) paradigm powers critical computational workloads from scientific simulations to large-scale machine learning training. Running on tens of thousands of processors, these systems are uniquely sensitive to performance variations that current observability approaches struggle to diagnose—post-hoc analysis of massive traces introduces latency and rigidity that obstructs timely diagnosis.

We present ORCA, a system for closed-loop BSP observability. ORCA enables operators to declaratively specify views materialized on demand via a tree-based overlay providing coordinated collection, in-situ SQL analytics, and timestep-consistent control. On a real AMR code at 4096 ranks, ORCA collects detailed traces at 1–2% overhead versus 56–81% for existing tracers, executes queries orders of magnitude faster, and reduces data volume by multiple orders of magnitude through in-situ analytics. In an agentic evaluation of the closed-loop workflow, a performance anomaly that took months to diagnose manually is localized in 27 minutes.

1 INTRODUCTION

The *bulk-synchronous parallel* (BSP) [1] paradigm powers critical computational workloads, from scientific simulations to large-scale machine learning training. These systems are uniquely fragile: at the scale of tens of thousands of processors, lockstep execution means a single straggler can block cluster-wide progress at every synchronization point. As ML training pushes BSP toward 100,000 GPUs and scientific workloads grow in tandem, the cost of undiagnosed performance pathologies has become economically untenable. Yet the causes are diverse and highly context-dependent [2–19] —load imbalance, poor overlap, hardware variability—varying with the cluster, workload, scale, and configuration. Metrics [20, 21] provide baseline visibility but rarely enough workload structure to attribute causes. Tracing [22–26] can capture the fine-grained patterns needed for diagnosis, but prevailing approaches—exhaustive collection to disk followed by expensive *extract-transform-load* (ETL) [10, 27] —cannot provide feedback at the speed problems demand. In our study of placement in AMR codes [10], isolating the root causes of several performance artifacts took months, while fixes typically took days. Rapid visibility into production pathologies remains one of the highest-yield paths to efficiency.

We argue for always-on, steerable observability as a production capability for BSP workloads—not just faster post-hoc analysis, but continuous visibility with the ability to escalate on demand. The evolution of distributed tracing [28–30] offers an instructive parallel. In microservices, tracing evolved to preserve causal relationships between requests—the unit of work in asynchronous systems—so tail latency is continuously monitored and attributable to specific services. In BSP, collectives induce causality: performance anomalies manifest as stragglers that stall collectives. And because BSP is a single coordinated workload, the system can unify observation and intervention—enabling operators to steer collection on demand, test hypotheses live, and act on feedback, all while the job runs. Real-time diagnostics are already proving valuable in ML training [14–19, 31] for problems ranging from straggler localization to training correctness, but each system implements a bespoke collection and analytics pipeline. A common observability substrate is needed for BSP workloads, providing low-overhead on-fabric transport, BSP-aware coordination semantics for telemetry aggregation and control, and programmable interfaces for analytics (§3).

We present a *tree-based overlay network* (TBON) for real-time observability and control of large-scale BSP applications, called ORCA (*Observability with Real-time Control and Aggregation*). ORCA treats telemetry as columnar streams, transformed via a SQL-based analytics interface called *OrcaFlow* en route to *sinks* such as dashboards or parallel filesystems. A key contribution is the *timestep dataframe* abstraction: building on the insight that global synchronization divides BSP telemetry into causally independent windows, ORCA transforms telemetry from a rank-major to a timestep-major representation (§4.1). An RDMA-based TBON-optimized transport (§4.2) enables low-overhead collection of these dataframes. They can then be transformed via SQL-based analytics, routed to a dashboard, or persisted as partitioned Parquet for zero-ETL postmortems. Distributed query planning with operator pushdown minimizes data movement across the overlay (§4.3). A control plane (§4.4) provides atomic and timestep-consistent control semantics, guaranteeing correctness for interventions such as new analytics, probes, parameter updates. ORCA combines a modern columnar analytics stack [32–34] with RDMA and BSP-aligned data and control paths to realize low-overhead, real-time, closed-loop feedback.

We evaluate ORCA on a real AMR code at up to 4096 CPU ranks.

- In conventional tracing workflows, ORCA incurs ~2% runtime overhead (vs 56–81% for state-of-the-art tracers), produces up to 42× smaller traces for equivalent telemetry, and enables up to 6900× faster analyses than existing trace formats (§6.1).
- In-situ analytics enable low-overhead needle-in-the-haystack workflows—outlier `MPI_Wait` calls are isolated in real-time at sub-2% runtime overhead, while discarding up to 99.999% of data at MPI ranks (§6.2).
- In an agentic evaluation of its closed-loop capabilities using an LLM agent as a proxy for a human operator, ORCA enables localizing a performance anomaly—the same bug from our AMR study that took months of manual investigation—in 27 minutes, while reducing data volume by up to 144× (§6.3).

More broadly, ORCA demonstrates that declarative SQL analytics can be embedded on the fabric alongside a live MPI application, enabling streaming analytics workflows beyond observability. ORCA is open source [35].

Conference'17, July 2017, Washington, DC, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

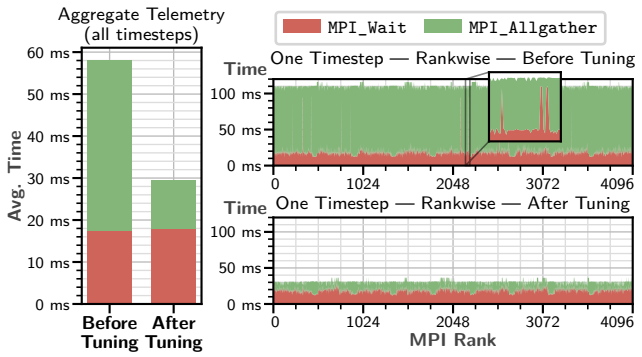


Figure 1: Telemetry from an AMR code where a volatile `MPI_Wait` precedes `MPI_Allgather`. Aggregate telemetry (left) misattributes straggler-induced `MPI_Wait` as elevated collective cost. Fine-grained telemetry (right) makes the cause obvious but requires joining all ranks' telemetry.

2 BACKGROUND AND MOTIVATION

Bulk synchronous parallel (BSP) [1] workloads in both scientific computing and machine learning training pose unique efficiency challenges. These jobs, increasingly spanning tens of thousands of nodes, execute in lockstep and synchronize frequently via collective communication. At these scales, BSP is uniquely fragile: a single straggler can block cluster-wide progress at synchronization points. Their root causes are highly context-dependent, ranging from load imbalance and suboptimal overlap to tuning-induced variability, and have been documented extensively [3–10, 13, 36–38]. Despite extensive studies, general-purpose mechanisms for operational visibility into BSP application performance remain elusive.

2.1 Motivation: Closing the Feedback Loop

Whereas monitoring targets *known unknowns* via a fixed set of metrics such as CPU and memory utilization, performance pathologies in BSP workloads are dominated by *unknown unknowns*. Observability [39–42] in this context is the ability to resolve them by constructing the right views from telemetry and refining them until causal attribution is possible.

Dominant observability paradigms for BSP workloads make this loop slow and rare. Diagnosing pathologies requires rerunning applications under progressively heavier diagnostics, from lightweight aggregates collected as profiles to fine-grained traces captured as per-rank event logs. At scale, these traces are too large to inspect visually, and programmatic analyses require expensive *extract-transform-load* (ETL) [27] before queries can be served. Additional instrumentation or interventions require repeating this workflow multiple times to isolate root causes and verify fixes. As a result, operators are forced to choose between expensive investigations and ad-hoc workarounds.

This gap motivates a different model: always-on observability as an operational capability, with relevant views derived in real-time, hypotheses tested live, and collection steered accordingly while the job is still running.

2.2 Case Studies from HPC and ML

2.2.1 Optimizing Placement in AMR. Our study of placement and load-balancing in *adaptive mesh refinement* (AMR) codes [10] illustrates this gap concretely. While widely known for load-imbalance challenges [43], we found that genuine load-imbalance was indistinguishable from other hardware, software, and tuning artifacts without fine-grained telemetry. Fig. 1 shows one such example: tail latency spikes in `MPI_Wait` that only manifested at scale would get attributed to collectives in aggregate telemetry. Each artifact took us months of iterative diagnoses, while mitigations would typically only take a few days.

Diagnosing each of theses issues required multiple iterations of fine-grained capture and ETL-heavy analysis over formats such as CSV. The mitigations identified were specific to our cluster and workload. Even after these artifacts were fixed, our placement policies demonstrated a fundamental tradeoff between compute load balance and communication locality — the right balance between the two varies across workloads, scales, and architectures. Translating these optimizations to production environments would require repeating this time-consuming exercise — underscoring the need for infrastructure that makes such investigations routine.

2.2.2 Parallels from the Pytorch Ecosystem. The PyTorch [44] ecosystem exhibits the same pattern. Kineto [24] captures fine-grained traces on demand as JSON. Dynolog [45] provides node-local control to trigger capture. *Holistic Trace Analysis* (HTA) [46] computes derived views such as activity breakdowns and straggler detection. Together, these form a closed loop, but with the same shortcomings we faced in our AMR study. HTA is a single-node Pandas [47] script that cannot scale to cluster-wide traces without expensive ETL. Dynolog provides best-effort actuation with no consistency guarantees, whereas reliable construction of views like Fig. 1 requires complete telemetry for each collective instance.

The wider ML training literature reinforces this pattern. Recent systems address pathologies such as straggler localization [14, 18, 19, 31, 48], fault diagnosis [16, 17], and training correctness [15], but each reimplements subsets of collection, aggregation, and control from scratch. A common substrate would allow detection strategies to be expressed as analytics queries over telemetry, composable and deployable without rebuilding the infrastructure beneath them.

2.3 Existing Approaches and Gaps

Monitoring. Monitoring enables basic visibility via metrics captured at sampled intervals, but lacks the workload context for causal attribution. Prometheus [20] is a time-series database for system counters, while LDMS [21] extends coverage for application-level statistics such as Kokkos [49]. They can not support synchronized on-demand collection necessary for views such as Fig. 1.

Profiling and Tracing. Tools such as TAU [22], Score-P [23], and Kineto [24] can provide both high-level aggregates or fine-grained telemetry via traces, but their workflows are post hoc by design. Traces can characterize short runs, but cannot provide always-on visibility over long-running jobs. In the absence of any real-time feedback to guide collection, the post hoc paradigm also incentivizes overcollection, leading to large datasets and long delays. Formats like OTF2 [50] are not designed for analytics, and visual inspection for anomalies is impractical at scale with billions of events.

Dataframe Analytics. Recent tools such as Caliper [26], DFTracer [25], Pipit [51], and Pytorch HTA [46] have adopted Pandas [47] and Dask-based [52] dataframe analytics interfaces to analyze telemetry, but require expensive ETL [27] to parse entire traces written as per-rank logs in plaintext formats — the working set for which scales with *ranks* × *timesteps*. Columnar formats like Parquet [33] offer efficient post-hoc analytics, but do not by themselves address overcollection, working set scaling, or real-time feedback, and remain largely unadopted in tracing workflows. In a simple internal port of HTA analyses from JSON to Parquet, we observed 50× faster data loading and 18× faster end-to-end analysis.

Streaming and In-situ Analytics. ADIOS [53] enables fixed-function in-situ workflows [54] on scientific data, typically run as another MPI application. MRNet [55, 56] is a tree-based overlay used as a reduction backend for commercial debuggers, but provides a compiled packet-based programming model. Neither enable runtime-programmable dataframe-style analytics needed for real-time BSP observability. Cloud streaming systems such as Flink [57] and Kafka [58] are designed for independent event streams over IP networks They lack fabric-aware transport and BSP-aligned coordinated collection, precluding views like Fig. 1.

2.4 The Missing Substrate

Diagnosing the root cause behind Fig. 1 required capabilities beyond what any single tool in the taxonomy provides. At node scale, eBPF [59] provides a template: dynamic probes, programmatic aggregation, low overhead in the common case with depth when needed. It enabled controlled and targeted access to telemetry within `MPI_Wait` after alternatives were exhausted. Hypotheses about interrupts and scheduler effects were tested and discarded in hours. But eBPF is node-local, and our workflow involved ad-hoc orchestration, uncoordinated collection, and manual post-hoc correlation. Extending this to workload scale—tens of thousands of BSP ranks—requires a substrate that provides the same depth on demand, treating observability not as distributed logging, but as constructing views directly from running applications via a programmable interface. The next section states the requirements such a substrate must satisfy.

3 REQUIREMENTS FOR CLOSED-LOOP OBSERVABILITY

We now state the key requirements for closed-loop observability. The goal is real-time feedback through interactive views over running applications, while retaining the ability to capture detailed traces when deeper investigation is required.

BSP-Aware Semantic Guarantees. BSP observability requires guarantees on both data and control paths. On the data path, this means coordinated collection: detecting the straggler at a barrier requires telemetry from all ranks for that barrier instance. Sampling yields uncoordinated subsets insufficient for root cause attribution. Control path semantics must similarly align with workload structure. Control inputs must be applied consistently and atomically at synchronization boundaries across all ranks. While uncoordinated updates may be tolerable in some cases, they break correctness for interventions such as hyperparameter changes — a timestep-consistent control plane generalizes across use cases.

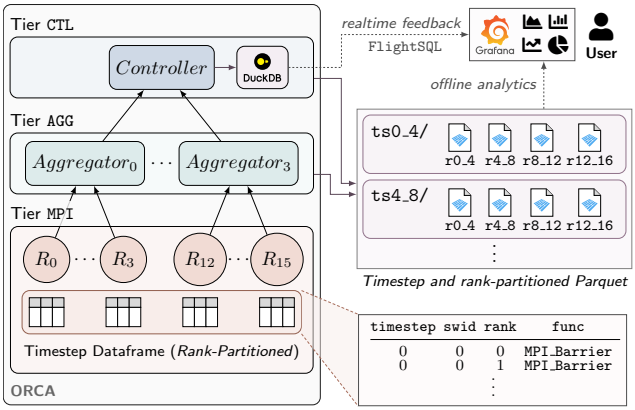


Figure 2: ORCA architecture. A three-tier TBON with application ranks (MPI) at the leaves, intermediate aggregators (AGG), and a controller (CTL) at the root. Telemetry originates as columnar timestep dataframes, is transformed by SQL plan operators at each tier, and persisted via DuckDB and Parquet sinks — enabling workflows from real-time feedback via lightweight metrics to zero-ETL post-hoc analytics.

Low-Cost, Low-Latency Feedback. Practical observability requires a view-driven model: emitting only what is needed to materialize requested views. First, overhead must be proportional to view complexity — lightweight aggregates for continuous monitoring should incur minimal cost, while detailed traces remain available on demand. Second, feedback must be fast, as the latency of feedback bounds the latency of intervention. Finally, the system must minimize induced perturbation — BSP workloads are jitter-sensitive, and overhead must scale with view complexity.

Declarative and Flexible Analytics. No predefined set of analyses can anticipate every pathology. Operators must be able to specify views and collection policies via a programmable interface at runtime, without recompilation or restart. The collection interface must also be source and schema-agnostic, as telemetry may be emitted by any layer of the runtime stack, including MPI, runtime libraries, or custom annotations.

These requirements are in tension. Coordinated semantics demand global completeness, low overhead requires not disrupting the application, and low feedback latency requires streaming to materialize views quickly instead of accumulating data locally. The programming model must be expressive enough for diagnostic queries, while remaining compatible with low-overhead execution. Our design combines OLAP primitives (Arrow [32], Parquet [33], DataFusion [34]) with RDMA for low-overhead transport, aligning data partitioning with synchronization boundaries, analytics with a reduction topology, and control with timestep semantics.

4 DESIGN

ORCA is a programmable *tree-based overlay network* (TBON) with a controller (CTL) at the root and aggregators (AGG) as the intermediate scale-out tier, attached to application ranks (MPI) as the leaf tier. CTL and AGG run on dedicated nodes outside the MPI domain and communicate with the application via on-fabric RPCs.

The TBON provides asynchronous data and control planes. Data originates as columnar Arrow streams at leaf ranks, flows through the TBON, and is persisted via sinks. Two sinks are currently implemented: Parquet [33] for high-volume telemetry, and DuckDB [60] for real-time metrics and dashboards. The architecture is shown in Fig. 2. We present ORCA as a progression of capabilities that successively shorten the observability feedback loop:

- The columnar data model (§4.1) and transport layer (§4.2) enable a workflow similar to conventional tracers, but more efficient to collect, store, and analyze.
- In-situ analytics (§4.3) and the OrcaFlow interface enable real-time feedback by transforming telemetry en-route to sinks. OrcaFlow chains SQL statements and map operators over multiple streams, routing results to configurable sinks.
- The control plane (§4.4) enables online interventions via TS2PC, a speculative variant of two-phase commit that ensures commands are atomically applied at all ranks at the same timestep.

4.1 Data Model: Timestep Dataframes

The logical unit of collection, analysis, and persistence in ORCA is a *timestep dataframe*. Applications emit named and *typed* streams with predefined Arrow schemas. Multiple streams may exist with different schemas, but tuples within a single stream must conform to a fixed schema. Each collective creates a new *synchronization window*, identified by a unique monotonic identifier (`swid`) that is assigned to all events within that window. A timestep is an application-level construct that may span multiple `swids`, as applications typically invoke multiple collectives within a timestep. Timestep dataframes comprise all tuples from all ranks within that timestep. These dataframes are logical constructs formed from rank-partitioned shards — they arrive at different tiers at different times via an asynchronous pipeline, but are analyzed and persisted as a single unit.

Typed Streams Enable Columnar Analytics. While JSON [61] and OTF2 [50] traces require expensive ETL before dataframe analytics, ORCA emits typed Arrow [32] data at the source and retains columnar structure throughout the pipeline — enabling zero-copy in-situ analytics and zero-ETL post-hoc analyses via Parquet [33] at rest.

Timesteps Are Causally Partitioned. Timestep dataframes enable efficient analytics by capturing the causal independence of BSP execution. At blocking collectives, all ranks wait for stragglers and execution resets. Each timestep is therefore an independent unit of analysis. Pathologies are attributed first to a straggler rank that delayed a collective, then to events in the dataframe that led to that straggler. The evolution of distributed tracing in loosely-coupled microservices — from treating traces as bags of events [28] to capturing causal edges between events via *request IDs* [29, 30] — reflects a similar evolution in request-based systems, but is a poor fit for BSP execution structure.

Instead of accumulating telemetry into per-rank logs, ORCA optimizes for analytics over timestep dataframes by partitioning telemetry along *(timestep, rank)* as the primary key before persisting as Parquet. Among HPC systems, Hatchet [62] postprocesses traces to build callflow graph indexes — a secondary index that is redundant when the primary key is effective. ORCA simply preserves the primary key for both in-situ and post-hoc analyses. As opposed to traces written as per-rank logs, this transposition from a rank-major to a timestep layout enables efficient post mortems, as data for any timestep only requires reading a fraction of the trace regardless of the workload duration (see Fig. 2).

4.2 TBON-Optimized Transport

ORCA's transport layer is designed to move Arrow dataframes asynchronously over the fabric across tiers. Overhead is minimized via an incast-optimized, receiver-driven mechanism that avoids synchronized pushes and overlaps data movement with application execution. As the CTL and AGG nodes are fully asynchronous and throughput-optimized, fabric QoS features [63] can be used to relegate ORCA traffic to a lower priority class, relying on idle fabric capacity for low-perturbation transfer. For telemetry persisted as Parquet, dedicated cores on AGG are used to compress and persist the data once collected.

The transport layer is agnostic to how telemetry is generated. Different profiling mechanisms accumulate telemetry into per-stream *SchemaCollectors* — local buffers that emit Arrow dataframes when flushed. After each timestep, all collectors on each rank are flushed to generate per-stream Arrow dataframes, which are then serialized into RDMA-registered memory. Ranks then send a metadata RPC to their aggregators containing RDMA descriptors for the serialized data. Synchronous end-of-timestep serialization into local buffers avoids jitter from mid-timestep flushes. Ranks then resume execution, while aggregators collect these dataframes via asynchronous RDMA GETs. As data is already prepartitioned by timestep and rank, the collection pipeline simply preserves this partitioning. While shuffling can create partitioning along arbitrary dimensions, ORCA does not currently implement it so as to maintain predictable ingestion throughput.

After multiple iterations, we developed three key mechanisms for stable operation over a TBON:

- Aggregators must issue RDMA GETs at controlled concurrency to prevent catastrophic throughput collapse, as RDMA NICs have limited resources for tracking active RDMA operations (as low as 1–2 on older fabrics). The pull-based model, which resembles receiver-driven flow control, lets aggregators stay in control of data volume and is essential for TBON incast topologies to work.
- Because the collection pipeline yields dataframes in timestep order, it is vulnerable to head-of-line blocking from missing shards, risking *out-of-memory* errors. We address this with a priority queue at aggregators that prioritizes lower timesteps.
- While the system must be provisioned with adequate bandwidth to ensure minimmal impact from backpressure, it is essential for stability. Aggregators communicate backpressure by withholding RPC ACKs, and ranks block if outstanding RPC count exceeds a configured limit.

4.3 In-Situ Analytics for Real-time Feedback

The Arrow-based timestep dataframe abstraction enables zero-copy in-situ SQL analytics at every tier of the TBON, enabling low-latency materialization of views and drastically reducing persisted data volume. OrcaFlow is a SQL-based programming interface that

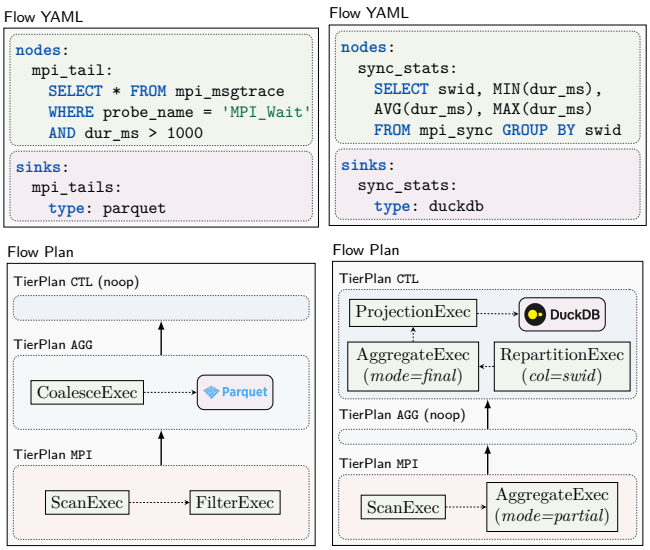


Figure 3: OrcaFlow compilation showing flows and their compiled tier plans. Flows chain SQL transforms over telemetry streams en route to configurable sinks. Both operators and sinks are placed automatically. (Left) A selective filter pushed to MPI, discarding most data at source. (Right) Aggregations also reduce data volume — pushed-down partial aggregations emit fixed-size intermediates regardless of input volume.

turns the TBON into a programmable reduction tree, transforming timestep dataframes en route to sinks — currently partitioned Parquet for offline analysis and DuckDB for real-time metrics. For analyses difficult to express in SQL, *map* operators allow invoking arbitrary precompiled kernels.

OrcaFlow is backed by a TBON-aware distributed planner and query engine built on Apache DataFusion. Each flow compiles into *tier plans* describing transforms to apply, data to forward, and sinks to execute locally. The planner automatically places both transforms and sinks, pushing operators that reduce data volume as close to the source as possible. MPI and AGG operate on rank-partitioned shards and execute per-partition operators: selection, projection, partial aggregation. Cross-partition operations like joins must run on CTL, the root aggregator. Sinks are placed on the lowest tier that produces their stream (no sinks on MPI, DuckDB only on CTL). Fig. 3 illustrates with two flows:

- *(Left)* A selective tracing flow for `MPI_Waits` > 10 ms pushes filters all the way to the MPI tier. Results are coalesced at AGG and written as Parquet. Most telemetry for pathologies like Fig. 1 gets discarded at source.
- *(Right)* An aggregation computing per-collective statistics splits into a partial aggregation at MPI and a final merge at CTL. Each leaf emits one row per group regardless of input volume — sketches and quantile digests fit the same model, producing fixed-size intermediates.

The programming model resembles batch analytics frameworks like Spark [64], but targets a multi-tier TBON, remains in-memory until sink ingestion, and avoids distributed shuffling to maintain predictable throughput. All tiers collect dataframes asynchronously

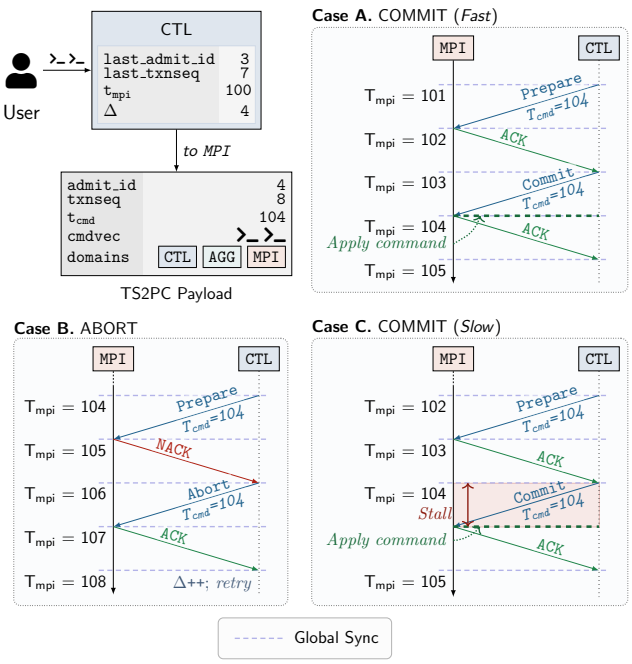


Figure 4: Timestep-linked two-phase commit (TS2PC). Commands are proposed for a future timestep $T_{cmd} = T_{mpi} + \Delta$. (A) Normal case: ranks agree and apply the command before executing T_{cmd}. (B) Abort: ranks have passed T_{cmd} and vote to abort, command is retried with a larger Δ. (C) Stall: ranks have prepared but not received commit/abort. Execution blocks until resolution to preserve correctness.

but analyze them in timestep order. Per-partition compute budget scales with MPI and AGG, while cross-partition work is limited to a single node (CTL). This suffices for residual work after filtering and pre-aggregation. Post-mortems remain the preferred workflow for expensive cross-partition analyses, but still benefit from ORCA's in-situ filtering, preaggregation, and partitioned Parquet.

4.4 Control Plane for Steerable Observability

A complete observability system must enable users to intervene in response to feedback, closing the loop between observation and action. In BSP workloads, this means steering collection and tuning application parameters at runtime in response to observed pathologies. Prior work has explored auto-tuning for specific use cases [65]. ORCA instead provides general mechanisms for timestep-consistent feedback and control, on which such policies can be built.

4.4.1 Requirements. The control plane must be asynchronous, atomic, and timestep-consistent. The controller is not part of the collective domain, so control must be asynchronous and must not stall application progress. Commands must be acknowledged by all ranks before being executed, and they must execute on all ranks at the same timestep to preserve BSP semantics.

4.4.2 The TS2PC Protocol. ORCA implements the control plane as a replicated state machine using *timestep-linked two-phase commit* (TS2PC). Commands are opaque payloads — the protocol only

decides when to commit and apply them. As the controller is not part of the MPI domain, it maintains a lagging view of the application's current timestep ($= T_{app}$) via asynchronous notifications from Rank 0. TS2PC proposes commands for a future timestep $T_{cmd} = T_{app} + \Delta$, where Δ is an automatically learned speculative margin. The TBON assists the protocol by broadcasting requests and reducing responses — any NAK in the subtree results in a single NAK at the controller. Fig. 4 illustrates the protocol via three cases.

- *Case A (Commit):* Ranks agree to commit if they have not yet reached T_{cmd}, and apply the command before executing T_{cmd}.
- *Case B (Abort):* If ranks are past T_{cmd}, they vote to abort. Δ is incremented and the transaction is automatically retried, quickly converging to the natural latency of the TBON.
- *Case C (Stall):* A subtlety arises when ranks have prepared but not yet received commit/abort by the time T_{cmd} arrives. Correctness requires stalling execution until commit/abort is received, so the prepared command can be applied before T_{cmd}.

As a result, TS2PC enables a control plane that is atomic, timestep-consistent, and asynchronous in the common case.

5 IMPLEMENTATION

The core ORCA framework consists of 40K lines of code (80% C++, 20% Rust). ORCA uses the Mercury [66] RPC library for on-fabric communication via libfabric [67] and UCX [68] providers. The Rust portion consists of OrcaFlow's DataFusion-based query planning and execution on each node, while the bulk of the functionality — bootstrap, transport, and control — lies on the C++ side.

5.1 Integration

ORCA provides two integration surfaces: the *OrcaClient* shim for applications, and *SchemaCollectors* for profiling interfaces.

OrcaClient. The *OrcaClient* shim links with the application and exposes a `PostTimestepAdvance()` hook, called after every timestep. ORCA can be activated at runtime by preloading the full library, otherwise these hooks become noops. `PostTimestepAdvance()` captures all data path work, including draining collectors, executing tier plans, dispatching data, and handling control messages.

SchemaCollectors. SchemaCollectors define stream schemas and capture events via `AddRow`, with one collector per stream. At the end of each timestep, ORCA drains collectors into per-stream Arrow dataframes and enqueues them for analysis. Our implementation currently packages basic collectors for MPI [69] and Kokkos [49]. Additional collectors may be defined by clients. Dynamic instrumentation via eBPF [59] and DynInst [70] remains future work.

5.2 Overlay: Bootstrap and Topology

The ORCA overlay bootstraps via a TCP-based discovery protocol for the initial handshake with the controller, and then switches to on-fabric RPCs once Mercury addresses are exchanged. Routes are then constructed as per the selected TBON topology, and available aggregators are automatically assigned to ranks. Each overlay entity is assigned a 32-bit ID called `ovid` to enable uniform addressing across all tiers. Multiple topologies are supported, including `DIRECT`, where all MPI ranks directly connect to the overlay, and `MPIREP`, where ranks choose node-local representatives to reduce aggregator incast. Additional topologies can be plugged into the framework. For simplicity, we only discuss the `DIRECT` topology in this paper.

5.3 Data Path

As discussed in §4.3, the data path is asynchronous but timestep-ordered — each tier executes its tier plans on dataframes in timestep order. The head-of-line blocking concerns are mitigated via a *(timestep, rank)*-ordered priority queue for pending GETs at AGG and CTL. Some key implementation aspects are described below:

- Each MPI rank emits one metadata RPC after every timestep regardless of data volume if ORCA is enabled. This is necessary for correctness, as higher tiers can only mark a timestep complete once the entire subtree has reported.
- Although Arrow analytics is zero-copy locally, ORCA performs a single memcpy from RDMA-registered memory to allow registered memory handles to be recycled faster.
- ORCA nodes are highly multithreaded and provision large thread pools for decode, compression, and Parquet I/O, as sustaining O(GB/s) requires multiple cores even with efficient compression codecs such as Snappy. While Arrow decode is highly efficient, costs still accumulate for complex flows at scale, with O(10K) data fragments received each second.
- Tier plan execution on aggregators and controller is multithreaded. The same rank-partitioning strategy used to shard timestep dataframes across tiers is used to partition shards across cores.

5.4 Control Plane: TS2PC

The TS2PC implementation consists of *inner* and *outer* layers. The inner layer enables a single transaction with a unique monotonic ID called `txnseq`, targeting a particular T_{cmd}.

Δ-learning. The outer TS2PC layer automatically retries commands with an incremented Δ in case of aborts. As a result, Δ quickly converges to the natural latency of the TBON.

Concurrency. Correctness necessitates that only one transaction be inflight at any given time, as intermittent failures can result in commands being committed out of order, potentially breaking correctness. However, a single transaction can contain an arbitrary number of commands applied as a batch.

Handling Pauses. The control plane supports pausing and resuming the application as primitives for more complex workflows. An interesting case emerges from the interaction of the paused state with TS2PC — when a resume is committed, it will be enqueued to be applied at a future timestep never reached by a paused application, creating a deadlock. Our solution recognizes that the paused state makes the application synchronous with the user and renders TS2PC unnecessary for timestep consistency. Commands in this state circumvent the TS2PC protocol and are executed directly.

Cross-tier Consistency. Commands such as flow updates trigger tier plan updates at all tiers, and must be applied at a consistent timestep across the entire TBON. This is challenging as the higher tiers are asynchronous and observe timestep at different times — these tiers are defined to observe a timestep when they process its data. Cross-tier consistency is supported via introducing control *domains* in the outer layer, allowing the entire overlay to be addressable via timestep-consistent control.

(1) Each tier corresponds to a domain, and each transaction contains a bitmask corresponding to the domains to which it applies. Flow updates, for example, apply to all domains, thus all bits are set.
(2) All transactions are committed by MPI regardless of the domain. The bitmask is only checked at *apply* time, and commands are committed only if the domain bit corresponding to that tier is set. As a result, MPI acts as the single source of truth on the application timestep.
(3) CTL and AGG act on control messages by *snooping* on commit messages. Commands applying to that tier are replicated internally before broadcasting the commit to the subtree. As the inner protocol guarantees correctness regardless of commits' arrival at MPI (Fig. 4, Case C) this ensures that commands for T_{cmd} are enqueued at all tiers before T_{cmd} is observed.

6 EVALUATION

We evaluate ORCA by progressively enabling its capabilities. As a passive telemetry collector, we compare runtime overhead (§6.1.1), storage efficiency (§6.1.2), and analytics performance (§6.1.3) against state-of-the-art tracers. We then demonstrate online analytics (§6.2) and control workflows (§6.3) — capabilities with no existing counterparts.

Hardware Setup. Experiments run on a 600-node research cluster with QLogic Truescale fabric (Intel Xeon E5-2670, 16 cores/node, 64GB RAM, 40 Gbps QLogic 7340 NICs) and a 20-node Lustre filesystem with 3GB/s aggregate bandwidth.

Software Setup. We use the Sedov Blast Wave 3D [71] AMR code from the Parthenon [72] and Phoebus [73] frameworks at 512–4096 CPU ranks, with 16 ranks/node. ORCA uses one aggregator node per 1024 ranks plus one controller node. All experiments use MVAPICH2 v2.3.7 [74] with a tuned PSM [75] library for our fabric. While simulation nodes use PSM, ORCA communication uses Mercury [66] over verbs because of fabric restrictions.

This setup represents a strenuous observability workload: CPU codes make every cycle visible as runtime overhead, the legacy fabric lacks QoS support and forces ORCA onto a non-native interface (verbs), and the AMR code's multiple communication patterns — multiple collectives within a timestep, halo exchanges, and frequent load balancing — make it highly jitter-sensitive.

6.1 Tracing Overhead and Efficiency

We compare ORCA with four state-of-the-art tracing tools — TAU [22], Score-P [23], Caliper [26], and DFTracer [25] — along three axes: runtime overhead, storage efficiency, and analytics performance.

Collection Profiles. We set up ORCA with two tracing profiles: a lightweight profile that only captures MPI collectives, and a detailed profile that also captures all MPI point-to-point messages and Kokkos events, except some extremely high-volume events such as `MPI_Test` and `MPI_Iprobe`. For ORCA, the lightweight profile measures the common-case overhead of always-on observability, while the detailed profile measures the worst-case overhead of capturing all MPI and Kokkos events. All other tracing tools are configured with an identical detailed collection profile. Unless otherwise specified, all experiments run for 2000 timesteps, and we report the average runtime from three runs. Unlike tracing tools, ORCA is deployed with additional resources — one aggregator node per 1024 ranks, plus one controller node.

6.1.1 Runtime Overhead. Fig. 5 compares the runtime overhead of ORCA and other tracing tools for 512–4096 ranks.

- ORCA remains within 0.8–2% overhead across all scales and profiles, while that of TAU and DFTracer degrades significantly — from 18–20% at 512 ranks to 56–81% at 4096 ranks (Score-P and Caliper discussed separately below).
- At 4096 ranks, ORCA moves ~550GB on-wire (~111GB after compression) over a shared fabric without QoS within 308 s at under 2% overhead — demonstrating that RDMA-based telemetry pipelines can coexist with tightly-coupled BSP communication.
- The marginal increase in ORCA overhead between the lightweight and detailed profiles is only ~0.5%, despite collecting over 200× more telemetry (524 MB vs 111 GB, respectively) — underscoring the efficiency of Arrow serialization and asynchronous RDMA-based collection.

Why ORCA Works. TAU and DFTracer degrade steadily because they flush trace buffers to storage as they fill, which produces uncoordinated flushing patterns. This, coupled with a kernel-based data path for storage, introduces jitter that compounds at scale. ORCA, in comparison, uses an inherently efficient format, serializes synchronously into RDMA-registered buffers, and transports via efficient RDMA GETs.

Caveats on Score-P and Caliper. Because of difficulties running them for the full 2000 timesteps, all numbers for Score-P and Caliper are extrapolated from 200 timesteps. Both tools are designed for short characterization workloads where data is buffered in memory and flushed once at the end. Score-P additionally requires two passes — the first to estimate buffer sizes — and frequently crashed with out-of-memory errors even with 64MB trace buffers. Caliper supported periodic flushing, but the path was buggy and performance was significantly degraded even after fixes.

6.1.2 Storage Efficiency. Fig. 6 compares the total size, and the per-record size of the traces emitted by ORCA and other tracers. ORCA writes Parquet, TAU and Score-P are configured for OTF2, DFTracer writes `jsonl`, and Caliper writes its custom plaintext `.cali` format. Key findings:

- ORCA consistently emits the smallest traces — 3.6–3.8× smaller than TAU, and up to 44.3× smaller than Caliper.
- The differences can be partly explained by format efficiency — OTF2, being a binary format, is only 1.4× larger than ORCA, while plaintext formats (`json` and `cali`) are 7–13× larger.

Record Amplification. Accounting for format efficiency, other tracing tools emit 2.5–3.3× more records than ORCA, despite the same collection profile.

(1) All tracers except DFTracer (and ORCA) generate separate entry and exit records for each traced function. ORCA and DFTracer generate one record, which contains both start time and duration. This is both more efficient and allows functions to be filtered by duration without requiring any preprocessing to pair the entry and exit records.
(2) Additionally, TAU incurs additional amplification from hard-coded derived events — for every MPI send and receive, it

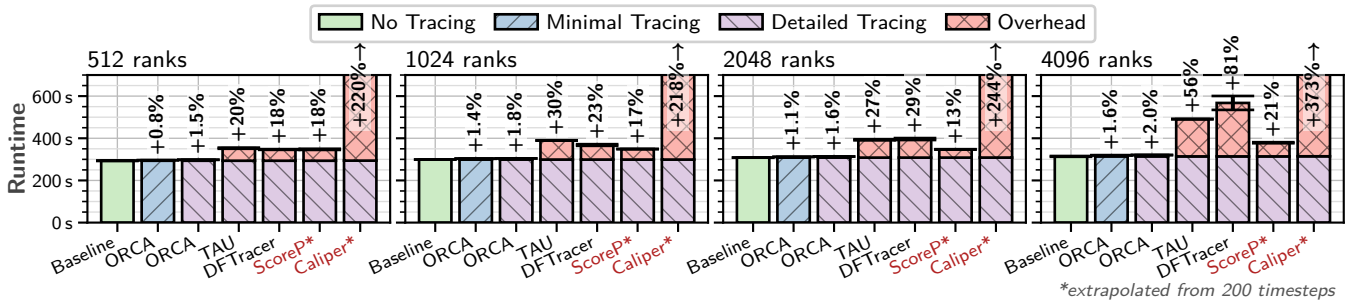


Figure 5: Runtime overhead for a detailed tracing profile on an AMR code at 512–4096 ranks. An additional minimal profile (collective-only) for ORCA shows always-on overhead. ORCA maintains 0.8–2% overhead across both profiles, while TAU and DFTracer degrade from 18–20% at 512 ranks to 56–81% at 4096 ranks. **extrapolated from 200 timesteps due to stability issues.*

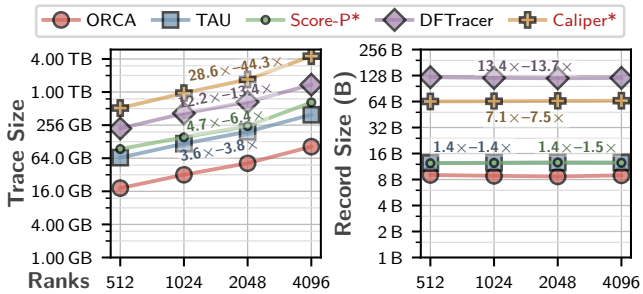


Figure 6: Trace size (left) and per-record size (right) comparison for detailed profile. ORCA traces are 3.6–44× smaller than other formats. Per-record size isolates format efficiency from record amplification — other tracers emit 2.5–3.3× more records than ORCA for the same collection profile.

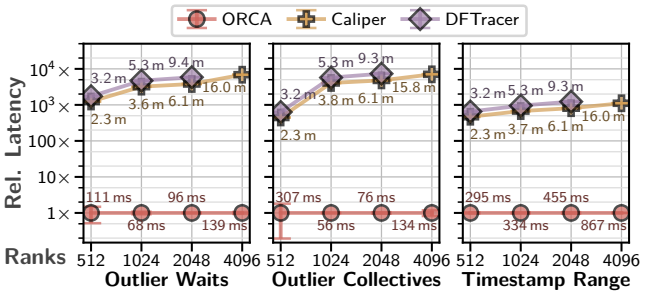


Figure 7: Post-mortem query performance. ORCA completes queries in hundreds of milliseconds, 3–4 orders of magnitude faster than DFTracer and Caliper. Performance for plaintext formats is dominated by parsing, not query complexity. TAU and Score-P are omitted as OTF2 is not a queryable format.

computes a function event, a message event, and an event for messages received during certain phases such as `MPI_Wait`.

(3) Flush-induced jitter, especially with Score-P and Caliper, creates stragglers that other ranks wait on by polling — generating additional events that inflate trace size.

Why ORCA Works. ORCA's storage efficiency stems from two design choices. First, Parquet yields an inherently compact representation through binary columnar layout, dictionary encoding, and Snappy compression. Second, separating derived events from collection — computing them via OrcaFlow rather than baking them into the tracer — prevents record amplification from creeping in as features accumulate. We discuss compression further in **??**.

6.1.3 Query Performance. We evaluate analytics performance using a suite of post-mortem queries (Fig. 7). All queries run on traces from a smaller 20-timestep run, executed cold with caches cleared, latencies averaged over 3 runs.

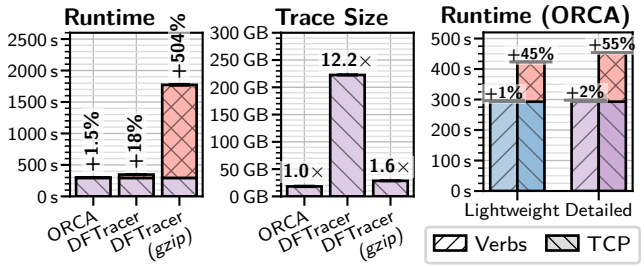
Queries. We use three queries of increasing complexity. *Outlier Waits* is a simple filter identifying `MPI_Wait` events exceeding 1 ms. *Outlier Collectives* aggregates collective events by `swid` and filters for outliers (max duration > 10 ms). *Timestamp Range* returns all events within the first second of execution—a query that might power a visualization frontend, and notably not a dimension ORCA partitions on.

Analytics Tools. All queries execute on a single 16-core node. DFTracer and Caliper use their bundled Python-based analytics libraries. ORCA uses unmodified Polars [76] with no partition awareness for simplicity— a handicapped configuration as the metadata from all Parquet files must be scanned for all queries. TAU and Score-P are excluded as OTF2 is not a queryable format.

Findings.

- ORCA queries complete in hundreds of milliseconds, 3–4 orders of magnitude faster than DFTracer and Caliper. The gap widens with scale.
- Query times for DFTracer and Caliper do not vary with query complexity. This is because performance is bound by parsing, not computation — plaintext formats must be parsed and loaded as dataframes before any queries can be computed.

Why ORCA Works. While all analytics-oriented tracers ultimately load data into dataframes, they persist traces as plaintext formats requiring expensive ETL to materialize dataframes before any query executes. ORCA preserves dataframe structure end-to-end, persisting directly as Parquet — enabling zero-ETL post-hoc analytics. Additionally, the layout confers two kinds of pruning: file-level pruning via (timestep, rank) partitioning, and intra-file pruning via rowgroup statistics. Our benchmark does not exploit the former — unmodified Polars with lazy execution must scan all file footers. Adopting Parquet-based lakehouse metadata formats [77, 78] would



(a) Compression impact. (b) Transport impact.

Figure 8: (Left) Compression reduces trace size but induces severe jitter on simulation ranks.ORCA avoids this by offloading compression to dedicated aggregators. (Right) TCP transport increases ORCA overhead from 1–2% to 45–55%. Arrow-RDMA co-design is essential for low overhead.

add a catalog layer over partitioned Parquet, making partition pruning automatically available via standard query engines [60, 76]. Even without partition pruning, ORCA outperforms by 3–4 orders of magnitude. Conventional formats offer no such pruning — as traces grow, larger analytics clusters must be provisioned for ETL before queries can begin.

6.1.4 Tradeoff Analysis. We isolate the impact of key architectural decisions using a 512-rank run (Fig. 8).

Compression. DFTracer supports gzip compression, which reduces its trace size from 12.2× that of ORCA to 1.6× (fig. 8a), but at the cost of increasing runtime overhead from 18% to 504%. Compression on MPI ranks is unviable because it competes with the application for CPU, amplifying jitter. ORCA offloads compression to dedicated aggregator cores, keeping it off the critical path. Additionally, compressed jsonl cannot be queried without full decompression, while ORCA's causal partitioning and Parquet's rowgroup structure reduce decompression overhead.

Transport Layer. Switching ORCA from `verbs` to the libfabric `TCP` provider increases overhead from 1–2% to 45–55% (fig. 8b). Even with the same Arrow-based data path, TCP suffers because retrieval incurs CPU overhead on simulation ranks, introducing jitter. This gap reflects the complementary nature of ORCA's transport choices: Arrow enables efficient serialization into contiguous RDMA-friendly buffers, and RDMA allows asynchronous retrieval with minimal CPU overhead while the application runs undisturbed. These results underscore the cost of TCP-based transport for tightly-coupled BSP workloads. GPU-based codes may tolerate the higher transport overhead, but event-based streaming tools still lack the columnar, causally-partitioned data path that enables ORCA's analytical efficiency.

6.2 ORCA-Native In-Situ Workflows

While ORCA can enable conventional tracing workflows, it is fundamentally designed for in-situ telemetry analytics — directly materializing views from a running application. Plan operators can be pushed all the way to the MPI tier, leveraging the application's own resources for data reduction at source. We now evaluate these capabilities, measuring the runtime overhead of operator pushdown, and its effectiveness at reducing captured data volume.

Flow	Query
Slow Wait Counts	`SELECT COUNT(*) FROM mpi_wait WHERE duration > 1ms` → DuckDB
Slow Wait Traces	`SELECT * FROM mpi_wait WHERE duration > 1ms` → Parquet
Load Imbalance	`SELECT rank, SUM(duration) FROM kokkos WHERE name IN <compute kernels> GROUP BY rank` → `stats` → Parquet `PERCENTILES(stats, [50,90,99])` → DuckDB

Table 1: In-situ workflows evaluated in Section 6.2, ranging from lightweight metrics and selective tracing on the communication path to aggregates over compute kernels.

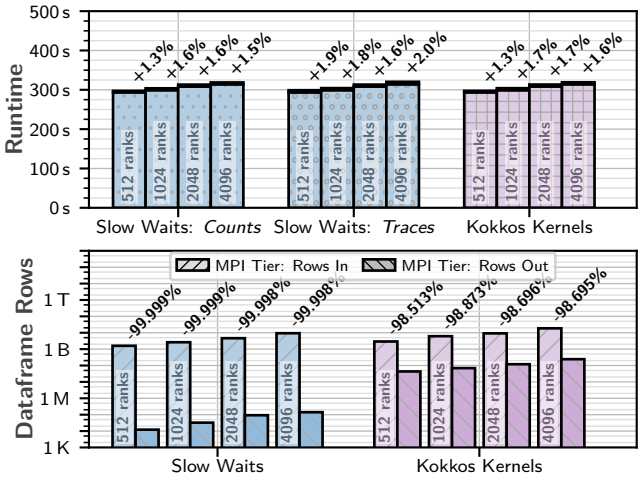


Figure 9: Runtime overhead (top) and MPI tier data reduction (bottom) for in-situ analytics with operator pushdown. Flows range from monitoring communication outliers to compute load balance. Pushdown discards 98.5–99.999% of data at sub-2% overhead, despite operators executing on MPI ranks.

Workflows. Table 1 shows three flows of increasing complexity, designed for AMR codes. The first flow demonstrates a monitoring workflow: measuring the volume of `MPI_Wait` exceeding 1ms, while the second flow persists all outlier events to Parquet for offline analysis. The third flow tracks compute load imbalance in AMR codes — it filters for known compute kernels, computes duration aggregates by rank, and streams details to Parquet and percentiles to DuckDB for real-time dashboards.

Observations. Fig. 9 shows the runtime overhead and the data reduction at MPI for the three flows.

- All workflows incur an overhead of ≤ 2% across all scales despite executing pushed-down operators on the MPI tier of a CPU simulation, illustrating the efficiency of Arrow-based analytics. Computation budget for pushdown only grows with modern GPU codes.
- Pushdown reduces transmitted data volume by 98.5% – 99.999%. Leveraging MPI nodes for analytics reduces fabric utilization, residual work at upper tiers, and shortens the feedback loop. The *capture-everything* nature of traditional tracing is redundant for

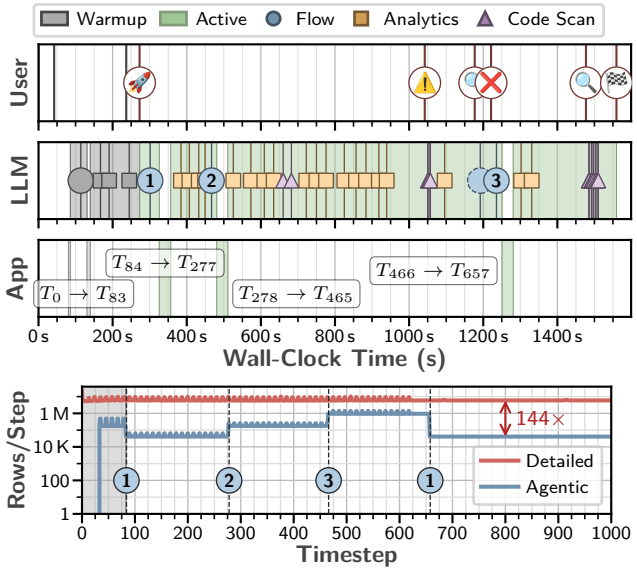


Figure 10: Visual transcript of an agentic debugging session demonstrating closed-loop capabilities. An LLM agent is tasked with localizing the anomaly from Fig. 1 using high-level metrics, progressively collecting targeted telemetry to identify the source. The agent operates largely autonomously, producing the complete call path in 27 minutes. Reverting to baseline monitoring reduces data volume by 144×.

monitoring and debugging—a small fraction of aggregated and filtered data suffices to materialize the required views.

- The `MPI_Wait` flows do not sample—they perform continuous predicate evaluation for every single `MPI_Wait`, emitting only outliers. Comprehensive continuous observability of this type is impractical without pushdown.

6.3 Closing the Loop: Agentic Debugging

While ORCA's control plane was designed for human operators, the emergence of LLM-based coding agents—capable of issuing commands and analyzing outputs—offers a way to validate the end-to-end feedback loop. To evaluate whether ORCA's interfaces are expressive enough for autonomous debugging, we use an LLM agent as a proxy for a human operator—tasking it with localizing the anomaly from Fig. 1, the same bug that took months to localize manually.

Setup. The anomaly is reproduced within the same AMR code used in previous experiments at 4096 ranks, using a placement policy that exacerbates the anomaly's frequency. The code is instrumented with a hierarchy of Kokkos region annotations, with additional annotations in functions of interest added by us. Claude Code (Opus 4.5) [79] ran on the head node with access to the ORCA controller via CLI and the codebase. The agent was tasked with monitoring collectives for outliers and progressively collecting only the Kokkos and MPI events necessary to localize the problem. The LLM issued all ORCA commands and performed all data analysis using off-the-shelf Polars [76] on the head node.

Timeline. Fig. 10 shows the debugging session: user and LLM interactions at top, per-timestep event volume at bottom. After a warmup phase, the LLM began the localization exercise at 272 s by monitoring collectives for outliers (1).

(1) At 384 s, the LLM analyzed collective percentiles and identified outlier stragglers. It installed a flow to collect Kokkos events with duration > 1 ms (2).
(2) At 510 s, it began investigating collected Kokkos events, correlated outliers with the collective pattern, and attributed the anomaly to a `map.clear()` call during AMR redistribution.
(3) At 1042 s, the user rejected the `map.clear()` hypothesis as an inadequate explanation for latencies between 50–100 ms. The LLM investigated further and identified that `map.clear()` triggered a buffer destructor calling `MPI_Wait`.
(4) At 1191 s, the user asked it to prove the `MPI_Wait` hypothesis using ORCA. The LLM proposed capturing all `MPI_Wait` events, but the user rejected this and asked for a duration-based filter. A revised flow with a 1 ms filter was accepted (3).
(5) At 1335 s, the LLM confirmed outlier waits as the root cause. Asked to trace the control flow leading to the anomalous `MPI_Wait`, it generated a report at 1605 s. The application then resumed with baseline collective monitoring (1).

Observations. The exercise validates ORCA's closed-loop capabilities. The agent autonomously correlated high-level telemetry with code, requested targeted detail, and traced the root cause to where instrumentation ended — completing in 27 minutes what took months of manual investigation. Reverting to baseline monitoring afterwards reduced data volume by 144×. User interventions were minor: rejecting an incomplete but accurate explanation and asking for basic duration-based filtering. The exercise was made easier with some additional instrumentation in relevant regions, but the result still demonstrates that ORCA's feedback loop enables online localization workflows as long as the right instrumentation is present. Support for dynamic instrumentation [59, 70] would close this gap, enabling investigations to continue into the MPI and fabric layers.

Manual investigation had previously traced the anomalous spikes to misplaced `ACK`s within the fabric driver, triggering an expensive timeout/retransmit path — subsequently fixed with an asynchronous drain queue. Beyond localization, ORCA supports always-on production monitoring of mitigations: the same signature that identified the anomaly can be deployed as a metric, enabling regressions to be detected immediately.

7 RELATED WORK

Tool Infrastructure and Tracing. MRNet [55] introduced TBONs for scalable tool infrastructure, but its compiled C++ filter model has not evolved with modern analytics. Fay [80] and Snicket [81] compile declarative queries into distributed probes that filter and aggregate at source. ORCA combines both: a TBON with Arrow interchange, declarative SQL, and BSP-aligned control. Dynamic instrumentation via eBPF [59] and DynInst [70] extends reach into runtime and kernel layers. Sketch-based telemetry reduction [82] aligns with ORCA's in-situ aggregation model.

ML Training Observability. Recent ML training systems address pathologies from straggler localization [12, 14, 18, 19, 31] to fault diagnosis [16, 17] to training correctness [15], each building application-specific collection and analysis pipelines. Mycroft [48] uses collective structure for real-time root-cause analysis, working backward from stalls to causes. Unlike ORCA's timestep dataframe abstraction, Mycroft analyzes approximate telemetry slices by querying a database, without completeness guarantees or in-situ reduction.

Unified Analytics. Arrow is emerging as common interchange for tabular HPC data —TileDB [83] and arrow-zarr [84] bridge array stores and SQL engines [34]. SmartNICs are being explored as execution targets for Arrow-based data pipelines [85, 86]— a declarative engine like ORCA's could offload plan fragments to such devices transparently. In-situ indexing systems like DeltaFS [87] and CARP [88] deploy shufflers for embedded in-situ hash and range-partitioning. ORCA's TBON could accommodate shuffling as an intra-tier operation. As these ecosystems converge, common infrastructure can subsume analytics workflows ranging from observability to checkpoint indexing to scientific queries.

8 CONCLUSIONS

BSP workloads require always-on observability as a production capability, capable of materializing actionable views and enabling rapid intervention. Traditional tracing works against this: the post-hoc paradigm incentivizes overcollection, requires large analytics clusters, and delays feedback. ORCA inverts this by pushing analytics to the source. Timestep dataframes and columnar representation enable multi-tier SQL analytics with operator pushdown—sub-2% overhead, up to 99.999% data reduction at source, agentic anomaly localization in minutes rather than months—all while contending for cluster resources with a live CPU simulation. GPU workloads and modern fabric QoS capabilities only increase the headroom available for such pushed-down workflows. Views materialize on demand, so only the stories—not the raw data—exit the fabric. The hero run need not be followed by a hero analysis.

REFERENCES

[1] Leslie G. Valiant. "A Bridging Model for Parallel Computation". In: *Commun. ACM* 33.8 (), pp. 103–111. ISSN: 0001-0782. DOI: 10.1145/79173.79181. URL: https://dl.acm.org/doi/10.1145/79173.79181.

[2] Xinhao Kong et al. "Collie: Finding Performance Anomalies in RDMA Subsystems". In: 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22), pp. 287–305. ISBN: 978-1-939133-27-4. URL: https://www.usenix.org/conference/nsdi22/presentation/kong.

[3] Zhibo Xuan et al. "Identifying Performance Inefficiencies of Parallel Program With Spatial and Temporal Trace Analysis". In: *IEEE Transactions on Parallel and Distributed Systems* 36.7 (), pp. 1387–1400. ISSN: 1558-2183. DOI: 10.1109/TPDS.2025.3566735. URL: https://ieeexplore.ieee.org/document/10982439/.

[4] Xin You et al. "GVARP: Detecting Performance Variance on Large-Scale Heterogeneous Systems". In: *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. SC24: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–16. DOI: 10.1109/SC41406.2024.00063. URL: https://ieeexplore.ieee.org/document/10793232/.

[5] Fabrizio Petrini, Darren J. Kerbyson, and Scott Pakin. "The Case of the Missing Supercomputer Performance: Achieving Optimal Performance on the 8,192 Processors of ASCI Q". In: *Proceedings of the 2003 ACM/IEEE Conference on Supercomputing*. SC '03: International Conference for High Performance Computing, Networking, Storage and Analysis. Phoenix AZ USA: ACM, p. 55. DOI: 10.1145/1048935.1050204. URL: https://dl.acm.org/doi/10.1145/1048935.1050204.

[6] Bilge Acun and Laxmikant V. Kale. "Mitigating Processor Variation through Dynamic Load Balancing". In: *2016 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 2016 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). Chicago, IL, USA: IEEE, pp. 1073–1076. ISBN: 978-1-5090-3682-0. DOI: 10.1109/IPDPSW.2016.74. URL: http://ieeexplore.ieee.org/document/7529982/.

[7] Bilge Acun, Phil Miller, and Laxmikant V. Kale. "Variation Among Processors Under Turbo Boost in HPC Systems". In: *Proceedings of the 2016 International Conference on Supercomputing*. ICS '16: 2016 International Conference on Supercomputing. Istanbul Turkey: ACM, pp. 1–12. ISBN: 978-1-4503-4361-9. DOI: 10.1145/2925426.2926289. URL: https://dl.acm.org/doi/10.1145/2925426.2926289.

[8] Haryadi S. Gunawi et al. "Fail-Slow at Scale: Evidence of Hardware Performance Faults in Large Production Systems". In: *ACM Transactions on Storage* 14.3 (), pp. 1–26. ISSN: 1553-3077, 1553-3093. DOI: 10.1145/3242086. URL: https://dl.acm.org/doi/10.1145/3242086.

[9] Brian Van Straalen et al. "Scalability Challenges for Massively Parallel AMR Applications". In: *2009 IEEE International Symposium on Parallel & Distributed Processing*. Distributed Processing (IPDPS). Rome, Italy: IEEE, pp. 1–12. ISBN: 978-1-4244-3751-1. DOI: 10.1109/IPDPS.2009.5161014. URL: http://ieeexplore.ieee.org/document/5161014/.

[10] Ankush Jain et al. "Lessons from Profiling and Optimizing Placement in AMR Codes". In: *2025 IEEE International Conference on Cluster Computing (CLUSTER)*. Edinburgh, UK: IEEE.

[11] Abhinav Bhatele et al. "The Case of Performance Variability on Dragonfly-based Systems". In: *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 896–905. DOI: 10.1109/IPDPS47924.2020.00096. URL: https://ieeexplore.ieee.org/document/9139880.

[12] Ziji Chen et al. *Detecting Anomalies in Machine Learning Infrastructure via Hardware Telemetry*. DOI: 10.48550/arXiv.2510.26008. arXiv: 2510.26008 [cs]. URL: http://arxiv.org/abs/2510.26008. Pre-published.

[13] Adithya Gangidi et al. "RDMA over Ethernet for Distributed Training at Meta Scale". In: *Proceedings of the ACM SIGCOMM 2024 Conference*. ACM SIGCOMM '24: ACM SIGCOMM 2024 Conference. Sydney NSW Australia: ACM, pp. 57–70. ISBN: 979-8-4007-0614-1. DOI: 10.1145/3651890.3672233. URL: https://dl.acm.org/doi/10.1145/3651890.3672233.

[14] Weihao Cui et al. *XPUTimer: Anomaly Diagnostics for Divergent LLM Training in GPU Clusters of Thousand-Plus Scale*. Version 1. DOI: 10.48550/arXiv.2502.05413. arXiv: 2502.05413. URL: http://arxiv.org/abs/2502.05413. Pre-published.

[15] Yuxuan Jiang et al. *Training with Confidence: Catching Silent Errors in Deep Learning Training with Automated Proactive Checks*. DOI: 10.48550/arXiv.2506.14813. arXiv: 2506.14813 [cs]. URL: http://arxiv.org/abs/2506.14813. Pre-published.

[16] Yangtao Deng et al. "Minder: Faulty Machine Detection for Large-scale Distributed Model Training". In: 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25), pp. 505–521. ISBN: 978-1-939133-46-5. URL: https://www.usenix.org/conference/nsdi25/presentation/deng.

[17] Jianbo Dong et al. "Evolution of Aegis: Fault Diagnosis for AI Model Training Service in Production". In: 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25), pp. 865–881. ISBN: 978-1-939133-46-5. URL: https://www.usenix.org/conference/nsdi25/presentation/dong.

[18] Zhiyi Yao et al. "Holmes: Localizing Irregularities in LLM Training with Mega-scale GPU Clusters". In: 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25), pp. 523–540. ISBN: 978-1-939133-46-5. URL: https://www.usenix.org/conference/nsdi25/presentation/yao.

[19] Tianyuan Wu et al. "GREYHOUND: Hunting Fail-Slows in Hybrid-Parallel Training at Scale". In: 2025 USENIX Annual Technical Conference (USENIX ATC 25), pp. 731–747. ISBN: 978-1-939133-48-9. URL: https://www.usenix.org/conference/atc25/presentation/wu-tianyuan.

[20] *Prometheus - Monitoring System & Time Series Database*. URL: https://prometheus.io/.

[21] Benjamin Schwaller. *Integrated System and Application Continuous Performance Monitoring and Analysis Capability (Final)*. SAND2021-11954, 1822583, 700207, SAND2021–11954, 1822583, 700207. URL: https://www.osti.gov/servlets/purl/1822583/.

[22] Sameer S. Shende and Allen D. Malony. "The Tau Parallel Performance System". In: *Int. J. High Perform. Comput. Appl.* 20.2 (), pp. 287–311. ISSN: 1094-3420. DOI: 10.1177/1094342006064482. URL: https://doi.org/10.1177/1094342006064482.

[23] Andreas Knüpfer et al. "Score-P: A Joint Performance Measurement Run-Time Infrastructure for Periscope, Scalasca, TAU, and Vampir". In: *Tools for High Performance Computing 2011*. Ed. by Holger Brunst et al. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 79–91. ISBN: 978-3-642-31475-9 978-3-642-31476-6. DOI: 10.1007/978-3-642-31476-6_7. URL: http://link.springer.com/10.1007/978-3-642-31476-6_7.

[24] *Pytorch Kineto*. pytorch. URL: https://github.com/pytorch/kineto.

[25] Hariharan Devarajan et al. "DFTracer: An Analysis-Friendly Data Flow Tracer for AI-Driven Workflows". In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*. SC '24. Atlanta, GA, USA: IEEE Press, pp. 1–24. ISBN: 979-8-3503-5291-7. DOI: 10.1109/SC41406.2024.00023. URL: https://dl.acm.org/doi/10.1109/SC41406.2024.00023.

[26] David Boehme et al. "Caliper: Performance Introspection for HPC Software Stacks". In: *SC '16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. SC '16: The International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 550–560. DOI: 10.1109/SC.2016.46. URL: https://ieeexplore.ieee.org/abstract/document/7877125.

[27] Izzet Yildirim et al. "IOMax: Maximizing Out-of-Core I/O Analysis Performance on HPC Systems". In: *Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*. SC-W '23. New York, NY, USA: Association for Computing Machinery, pp. 1209–1215. ISBN: 979-8-4007-0785-8. DOI: 10.1145/3624062.3624191. URL: https://dl.acm.org/doi/10.1145/3624062.3624191.

[28] Paul Barham et al. "Using Magpie for Request Extraction and Workload Modelling". In: 6th Symposium on Operating Systems Design & Implementation (OSDI 04). URL: https://www.usenix.org/conference/osdi-04/using-magpie-request-extraction-and-workload-modelling.

[29] Rodrigo Fonseca et al. "{X-Trace}: A Pervasive Network Tracing Framework". In: 4th USENIX Symposium on Networked Systems Design & Implementation (NSDI 07). URL: https://www.usenix.org/conference/nsdi-07/x-trace-pervasive-network-tracing-framework.

[30] Benjamin H. Sigelman et al. "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure". In: (). URL: https://research.google.com/pubs/pub36356.html?ref=highscalability.com.

[31] Yu Guan et al. *PerfTracker: Online Performance Troubleshooting for Large-scale Model Training in Production*. Version 2. DOI: 10.48550/arXiv.2506.08528. arXiv: 2506.08528 [cs]. URL: http://arxiv.org/abs/2506.08528. Pre-published.

[32] *Apache Arrow*. Apache Arrow. URL: https://arrow.apache.org/.

[33] *Apache Parquet*. Apache Parquet. URL: https://parquet.apache.org/docs/overview/.

[34] Andrew Lamb et al. "Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine". In: *Companion of the 2024 International Conference on Management of Data*. SIGMOD/PODS '24: International Conference on Management of Data. Santiago AA Chile: ACM, pp. 5–17. ISBN: 979-8-4007-0422-2. DOI: 10.1145/3626246.3653368. URL: https://dl.acm.org/doi/10.1145/3626246.3653368.

[35] Anonymous Authors. *ORCA: Observability with Real-time Control and Aggregation*.

[36] Brian Kocoloski and John Lange. "Varbench: An Experimental Framework to Measure and Characterize Performance Variability". In: *Proceedings of the 47th International Conference on Parallel Processing*. ICPP 2018: 47th International Conference on Parallel Processing. Eugene OR USA: ACM, pp. 1–10. ISBN: 978-1-4503-6510-9. DOI: 10.1145/3225058.3225125. URL: https://dl.acm.org/doi/10.1145/3225058.3225125.

[37] Ruiming Lu et al. "PERSEUS: A Fail-Slow Detection Framework for Cloud Storage Systems". In: ().

[38] Orcun Yildiz et al. "On the Root Causes of Cross-Application I/O Interference in HPC Storage Systems". In: *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 750–759. DOI: 10.1109/IPDPS.2016.50. URL: https://ieeexplore.ieee.org/document/7516071.

[39] Charity Majors. *Observability: What's in a Name?* Honeycomb. URL: https://www.honeycomb.io/blog/observability-whats-in-a-name.

[40] Majors, Charity. *Observability Is a Many-Splendored Definition*. charity.wtf. URL: https://charity.wtf/2020/03/03/observability-is-a-many-splendored-thing/.

[41] *Observability Primer*. OpenTelemetry. URL: https://opentelemetry.io/docs/concepts/observability-primer/.

[42] Cindy Sridharan. *Distributed Systems Observability*. ISBN: 978-1-4920-3343-1. URL: https://www.oreilly.com/library/view/distributed-systems-observability/9781492033431/.

[43] Anshu Dubey et al. "A Survey of High Level Frameworks in Block-Structured Adaptive Mesh Refinement Packages". In: *Journal of Parallel and Distributed Computing* 74.12 (), pp. 3217–3227. ISSN: 07437315. DOI: 10.1016/j.jpdc.2014.07.001. URL: https://linkinghub.elsevier.com/retrieve/pii/S0743731514001178.

[44] Adam Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc. URL: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

[45] *Facebookincubator/Dynolog*. Meta Incubator. URL: https://github.com/facebookincubator/dynolog.

[46] *Facebookresearch/HolisticTraceAnalysis*. Meta Research. URL: https://github.com/facebookresearch/HolisticTraceAnalysis.

[47] *Pandas - Python Data Analysis Library*. URL: https://pandas.pydata.org/.

[48] Yangtao Deng et al. "Mycroft: Tracing Dependencies in Collective Communication Towards Reliable LLM Training". In: *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. SOSP '25. New York, NY, USA: Association for Computing Machinery, pp. 254–269. ISBN: 979-8-4007-1870-0. DOI: 10.1145/3731569.3764848. URL: https://doi.org/10.1145/3731569.3764848.

[49] Christian R. Trott et al. "Kokkos 3: Programming Model Extensions for the Exascale Era". In: *IEEE Transactions on Parallel and Distributed Systems* 33.4 (), pp. 805–817. ISSN: 1558-2183. DOI: 10.1109/TPDS.2021.3097283. URL: https://ieeexplore.ieee.org/document/9485033.

[50] Eschweiler Dominic et al. "Open Trace Format 2: The Next Generation of Scalable Trace Formats and Support Libraries". In: *Advances in Parallel Computing*. IOS Press. DOI: 10.3233/978-1-61499-041-3-481. URL: https://www.medra.org/servlet/aliasResolver?alias=iospressISSNISBN&issn=0927-5452&volume=22&spage=481.

[51] Abhinav Bhatele et al. *Pipit: Scripting the Analysis of Parallel Execution Traces*. Version 2. DOI: 10.48550/arXiv.2306.11177. arXiv: 2306.11177 [cs]. URL: http://arxiv.org/abs/2306.11177. Pre-published.

[52] Matthew Rocklin. "Dask: Parallel Computation with Blocked Algorithms and Task Scheduling". In: Python in Science Conference. Austin, Texas, pp. 126–132. DOI: 10.25080/Majora-7b98e3ed-013. URL: https://doi.curvenote.com/10.25080/Majora-7b98e3ed-013.

[53] William F. Godoy et al. "ADIOS 2: The Adaptable Input Output System. A Framework for High-Performance Data Management". In: *SoftwareX* 12 (). ISSN: 2352-7110. DOI: 10.1016/j.softx.2020.100561. URL: https://www.softxjournal.com/article/S2352-7110(19)30256-0/fulltext.

[54] Greg Eisenhauer et al. *Streaming Data in HPC Workflows Using ADIOS*. DOI: 10.48550/arXiv.2410.00178. arXiv: 2410.00178 [cs]. URL: http://arxiv.org/abs/2410.00178. Pre-published.

[55] P.C. Roth, D.C. Arnold, and B.P. Miller. "MRNet: A Software-Based Multicast/Reduction Network for Scalable Tools". In: *SC '03: Proceedings of the 2003 ACM/IEEE Conference on Supercomputing*. SC '03: Proceedings of the 2003 ACM/IEEE Conference on Supercomputing, pp. 21–21. DOI: 10.1145/1048935.1050172. URL: https://ieeexplore.ieee.org/abstract/document/1592924.

[56] William R. Williams et al. "Dyninst and MRNet: Foundational Infrastructure for Parallel Tools". In: *Tools for High Performance Computing 2015*. Ed. by Andreas Knüpfer et al. Cham: Springer International Publishing, pp. 1–16. ISBN: 978-3-319-39589-0. DOI: 10.1007/978-3-319-39589-0_1.

[57] Paris Carbone et al. "Apache Flink™: Stream and Batch Processing in a Single Engine". In: *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering* 36.4 (), pp. 28–38.

[58] Jay Kreps, Neha Narkhede, Jun Rao, et al. "Kafka: A Distributed Messaging System for Log Processing". In: *Proceedings of the NetDB*. Vol. 11. 2011. Athens, Greece, pp. 1–7.

[59] *What Is eBPF? An Introduction and Deep Dive into the eBPF Technology*. URL: https://ebpf.io/what-is-ebpf/.

[60] Mark Raasveldt and Hannes Mühleisen. "DuckDB: An Embeddable Analytical Database". In: *Proceedings of the 2019 International Conference on Management of Data*. SIGMOD/PODS '19: International Conference on Management of Data. Amsterdam Netherlands: ACM, pp. 1981–1984. ISBN: 978-1-4503-5643-5. DOI: 10.1145/3299869.3320212. URL: https://dl.acm.org/doi/10.1145/3299869.3320212.

[61] *Catapult - The Trace-Event File Format*. URL: https://chromium.googlesource.com/catapult/+/HEAD/docs/trace-event-format.md.

[62] Abhinav Bhatele, Stephanie Brink, and Todd Gamblin. "Hatchet: Pruning the Overgrowth in Parallel Profiles". In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. SC '19: The International Conference for High Performance Computing, Networking, Storage, and Analysis. Denver Colorado: ACM, pp. 1–21. DOI: 10.1145/3295500.3356219. URL: https://dl.acm.org/doi/10.1145/3295500.3356219.

[63] Daniele De Sensi et al. "An In-Depth Analysis of the Slingshot Interconnect". In: *SC20: International Conference for*

High Performance Computing, Networking, Storage and Analysis, pp. 1–14. DOI: 10.1109/SC41405.2020.00039. arXiv: 2008.08886 [cs]. URL: http://arxiv.org/abs/2008.08886.

[64] Matei Zaharia et al. "Spark: Cluster Computing with Working Sets". In: *Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing*. HotCloud'10. USA: USENIX Association, p. 10.

[65] Shajulin Benedict, Ventsislav Petkov, and Michael Gerndt. "PERISCOPE: An Online-Based Distributed Performance Analysis Tool". In: *Tools for High Performance Computing 2009*. Ed. by Matthias S. Müller et al. Berlin, Heidelberg: Springer, pp. 1–16. ISBN: 978-3-642-11261-4. DOI: 10.1007/978-3-642-11261-4_1.

[66] Jerome Soumagne et al. "Mercury: Enabling Remote Procedure Call for High-Performance Computing". In: *2013 IEEE International Conference on Cluster Computing (CLUSTER)*. 2013 IEEE International Conference on Cluster Computing (CLUSTER). Indianapolis, IN, USA: IEEE, pp. 1–8. ISBN: 978-1-4799-0898-1. DOI: 10.1109/CLUSTER.2013.6702617. URL: http://ieeexplore.ieee.org/document/6702617/.

[67] Paul Grun et al. "A Brief Introduction to the OpenFabrics Interfaces - A New Network API for Maximizing High Performance Application Efficiency". In: *2015 IEEE 23rd Annual Symposium on High-Performance Interconnects*. 2015 IEEE 23rd Annual Symposium on High-Performance Interconnects, pp. 34–39. DOI: 10.1109/HOTI.2015.19. URL: https://ieeexplore.ieee.org/document/7312664.

[68] Pavel Shamis et al. "UCX: An Open Source Framework for HPC Network APIs and Beyond". In: *2015 IEEE 23rd Annual Symposium on High-Performance Interconnects*. 2015 IEEE 23rd Annual Symposium on High-Performance Interconnects, pp. 40–43. DOI: 10.1109/HOTI.2015.13. URL: https://ieeexplore.ieee.org/document/7312665.

[69] "MPI: A Message-Passing Interface Standard". In: ().

[70] Andrew R. Bernat and Barton P. Miller. "Anywhere, Any-Time Binary Instrumentation". In: *Proceedings of the 10th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools*. PASTE '11. New York, NY, USA: Association for Computing Machinery, pp. 9–16. ISBN: 978-1-4503-0849-6. DOI: 10.1145/2024569.2024572. URL: https://dl.acm.org/doi/10.1145/2024569.2024572.

[71] Sedov, Leonid Ivanovich. *Similarity and Dimensional Methods in Mechanics*. URL: http://archive.org/details/1-i-sedov.-similarity-and-dimensional-methods-in-mechanics.

[72] Philipp Grete et al. "Parthenon—a Performance Portable Block-Structured Adaptive Mesh Refinement Framework". In: *The International Journal of High Performance Computing Applications* 37.5 (), pp. 465–486. ISSN: 1094-3420. DOI: 10.1177/10943420221143775. URL: https://doi.org/10.1177/10943420221143775.

[73] Brandon Barker et al. *Phoebus: Performance Portable GRMHD for Relativistic Astrophysics*. Version 2. DOI: 10.48550/arXiv.2410.09146. arXiv: 2410.09146 [astro-ph]. URL: http://arxiv.org/abs/2410.09146. Pre-published.

[74] Dhabaleswar Kumar Panda et al. "The MVAPICH Project: Transforming Research into High-Performance MPI Library for HPC Community". In: *Journal of Computational Science*. Case Studies in Translational Computer Science 52 (), p. 101208. ISSN: 1877-7503. DOI: 10.1016/j.jocs.2020.101208. URL: https://www.sciencedirect.com/science/article/pii/S1877750320305093.

[75] *PSM: Performance Scaled Messaging*. Intel® Corporation. URL: https://github.com/intel/psm.

[76] *Polars*. URL: https://www.pola.rs/.

[77] *Apache Iceberg - Apache Iceberg™*. URL: https://iceberg.apache.org/.

[78] Michael Armbrust et al. "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores". In: *Proceedings of the VLDB Endowment* 13.12 (), pp. 3411–3424. ISSN: 2150-8097. DOI: 10.14778/3415478.3415560. URL: https://dl.acm.org/doi/10.14778/3415478.3415560.

[79] *Claude Code Overview*. Claude Code Docs. URL: https://code.claude.com/docs/en/overview.

[80] Úlfar Erlingsson et al. "Fay: Extensible Distributed Tracing from Kernels to Clusters". In: *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*. SOSP '11: ACM SIGOPS 23nd Symposium on Operating Systems Principles. Cascais Portugal: ACM, pp. 311–326. ISBN: 978-1-4503-0977-6. DOI: 10.1145/2043556.2043585. URL: https://dl.acm.org/doi/10.1145/2043556.2043585.

[81] Jessica Berg et al. "Snicket: Query-Driven Distributed Tracing". In: *Proceedings of the Twentieth ACM Workshop on Hot Topics in Networks*. HotNets '21: The 20th ACM Workshop on Hot Topics in Networks. Virtual Event United Kingdom: ACM, pp. 206–212. ISBN: 978-1-4503-9087-3. DOI: 10.1145/3484266.3487393. URL: https://dl.acm.org/doi/10.1145/3484266.3487393.

[82] Milind Srivastava and Vyas Sekar. "Circa: Re-imagining Network Telemetry from an Approximation-First Perspective". In: *Proceedings of the ACM SIGCOMM 2024 Conference: Posters and Demos*. ACM SIGCOMM Posters and Demos '24. New York, NY, USA: Association for Computing Machinery, pp. 57–59. ISBN: 979-8-4007-0717-9. DOI: 10.1145/3672202.3673742. URL: https://dl.acm.org/doi/10.1145/3672202.3673742.

[83] *TileDB • Designed for Discovery*. TileDB. URL: https://tiledb.com/.

[84] *Datafusion-Contrib/Arrow-Zarr*. datafusion-contrib. URL: https://github.com/datafusion-contrib/arrow-zarr.

[85] Craig Ulmer et al. "Extending Composable Data Services into SmartNICs". In: *2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), pp. 953–959. DOI: 10.1109/IPDPSW59300.2023.00158. URL: https://ieeexplore.ieee.org/document/10196516.

[86] Jianshen Liu et al. *Processing Particle Data Flows with SmartNICs*. arXiv: 2210.06827 [cs]. URL: http://arxiv.org/abs/2210.06827. Pre-published.

[87] Qing Zheng et al. "Scaling Embedded In-Situ Indexing with DeltaFS". In: *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*. SC18: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 30–44. DOI: 10.1109/SC.2018.00006. URL: https://ieeexplore.ieee.org/document/8665820.

[88] Ankush Jain et al. "CARP: Range Query-Optimized Indexing for Streaming Data". In: *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. Supercomputing.